

LSIN200A - Cours 7

Coline Gianfrotta

coline.gianfrotta@ens.uvsq.fr

2 avril 2026

La bibliothèque numpy

- La bibliothèque numpy permet de faire des calculs numériques en Python
- Pour l'utiliser, il faut l'importer avec l'instruction suivante:

```
import numpy as np
```

Exemple

```
print(np.pi) ## nombre pi
print(np.exp(1)) ## fonction exponentielle
print(np.cos(0.1)) ## cosinus d'un angle en radians
```

La bibliothèque numpy: `np.array`

- C'est un outil performant pour la manipulation de tableaux à plusieurs dimensions, avec le type d'objets `np.array`
- Contrairement à la liste en Python, le tableau `np.array` ne peut contenir que des éléments de même type
- Mais, il permet d'effectuer rapidement et simplement de nombreuses opérations mathématiques

Exemple

```
## On peut créer un array à partir d'une liste
t = np.array([1,2,3,4])
print(np.mean(t)) # moyenne
print(np.median(t)) # médiane
```

Création d'un numpy array

- à partir d'une liste

```
liste = [3, 5, 7]
t1 = np.array(liste) # 1 dimension
t2 = np.array([1,2,3,4]) # 1 dimension

# 1 dimension à partir d'une liste en compréhension
t3 = np.array([k**2 for k in range(-3, 4)])

t4 = np.array([[1,2],[3,4]]) # 2 dimensions
```

- à partir de fonctions existantes (np.empty, np.zeros, np.ones, np.arange, ...)

```
t1 = np.zeros(2) # 1 dimension
t2 = np.ones((2,2)) # 2 dimensions
t3 = np.empty((1,2)) # 2 dimensions, valeurs non initialisées
t4 = np.arange(1, 5, 2) # 1 dimension, equivalent à range
                        # sur les listes
```

Caractéristiques d'un numpy array

- Pour un numpy array noté `t`, on peut avoir accès à:
 - son nombre de dimensions (`t.ndim`)
 - son nombre d'éléments (`t.size`)
 - le type des éléments le contenant (`t.dtype`)
 - sa forme, c'est-à-dire le nombre d'éléments dans chaque dimension (`t.shape`)

Exemple

```
t = np.array([[1,2], [3,4], [5,6]])  
print(t.shape) # renvoie le tuple (3,2)  
print(t.size) # renvoie 6  
print(t.ndim) # renvoie 2
```

Accès aux éléments

- L'accès aux éléments se fait de la même façon pour les listes.
- Une particularité cependant lorsqu'il y a 2 dimensions: pour l'élément en i ème ligne et j ème colonne, il faut écrire `t[i,j]` et non `t[i][j]`.

Exemple

```
t = np.array([[1,2], [3,4], [5,6]])  
print(t[0]) # 1ère ligne  
print(t[0,1])  
print(t[1:,:1])
```

Copie et modification de numpy array

- Comme pour les listes, il faut faire attention lorsqu'on veut copier un numpy array

```
t = np.array([[1,2], [3,4], [5,6]])  
t2 = t[0] # t2 contient la première ligne de t  
t2[0] = 3 # t2 est modifiée mais t l'est aussi
```

- Pour pallier ce problème, il faut utiliser la méthode copy.

```
t2 = t[0].copy() # t2 contient une copie de la première  
                 # ligne de t  
t2[0] = 3 # t2 est modifiée mais t ne l'est pas
```

Opérations sur les numpy arrays

- Lorsque le numpy array contient des nombres, il est possible de faire des opérations vectorielles ou matricielles, de manière beaucoup plus rapide qu'avec le type list

```
A = np.array([[1,2], [3,4], [5,6]])  
print(A + A) # somme de deux matrices  
print(A * 2) # multiplication par une constante  
print(A.T) # transposition
```


Multiplication matricielle

- Attention, l'opérateur `*` ne permet pas de faire un produit matriciel, mais un produit élément par élément
- Pour faire un produit matriciel, on peut utiliser la méthode `dot` ou l'opérateur `@`

```
t1 = np.array([[1,2], [3,4]])
t2 = np.array([[3,4], [1,6]])
print(t1 * t2) ## produit scalaire, élément par élément
print(t1.dot(t2)) ## produit matriciel
print(t1 @ t2) ## produit matriciel
                # (il faut que t1.shape[1] soit
                # égal à t2.shape[0])
```

Autres opérations

- La bibliothèque numpy permet également de faire des opérations élémentaires sur les numpy arrays:
 - `np.min`
 - `np.max`
 - `np.mean`
 - `np.std` (écart-type, standard deviation en anglais)
 - ...
- Dans le cas d'un array à 2 dimensions (représentant une matrice), on peut faire ces calculs par ligne ou par colonne, avec le mot-clé `axis`

Exemple

```
t = np.array([[1,4], [2,3]])
print(np.min(t)) ## donne la valeur minimum de la matrice
print(np.min(t, axis=0)) ## donne les valeurs minimum
                        # par colonne dans la matrice
print(np.min(t, axis=1)) ## donne les valeurs minimum
                        # par ligne dans la matrice
```

Résoudre un système linéaire avec numpy

- La fonction `np.linalg.solve` permet de résoudre un système linéaire

Par exemple, si on veut résoudre le système $Ax = b$ suivant:

$$\begin{pmatrix} 1 & 2 \\ 3 & 5 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \quad (1)$$

```
A = np.array([[1, 2], [3, 5]])  
b = np.array([1, 2])  
x = np.linalg.solve(A, b)  
print(x)
```